

CODE SPORT



In this month's column, we discuss some more computer science interview questions.

Let us continue to look at more computer science (CS) interview questions that cover algorithms, data structures, machine learning (ML) and natural language processing (NLP). Interviews often vary across companies, but typically focus on certain fundamental concepts and topics that every computer scientist must be familiar with. That's why we often focus on the basics of algorithms, ML and NLP in this column, rather than discussing specific in-depth questions, which are rare in CS interviews. However, in this month's column, we will include a few questions that are on in-depth topics, just to give readers an idea. But this is not the norm in most interviews, which typically cover the fundamental concepts. So focus 80-90 per cent of interview preparation on fundamental questions and the remaining 10-20 per cent on specific in-depth areas, based on the company you are planning to join. Let us get started with our set of questions for this month.

1. Most of you are familiar with supervised machine learning approaches. The fundamental concept of learning in a supervised approach is to use the loss function, which characterises the difference between the predicted output and the expected output, and use the derivative of this function with respect to the weights to adjust these in each step of learning. What are some of the common loss functions you are familiar with? What is the loss function you will select for a linear regression problem? What is the loss function you will select for a binary classification problem typically?
2. What is the loss function for logistic regression?
3. How does the logistic regression loss function change when you use L1 regularisation and L2 regularisation?
4. Why does L1 regularisation lead to sparse solution for the weights, resulting in feature selection? (With certain weights being driven down to zero, those features are dropped from the classifier.)
5. Let us assume that you have been asked to design a deep neural network for a clustering problem. You are given a set of points and asked to cluster them, using a deep neural network. In this case, there is no ground truth; namely, you don't have the cluster labels for each point. Is it possible to come up with a loss function for this problem? If yes, explain how. If no, justify why.
6. You are given a corpus of sentences along with the labels for the sentiment associated with each sentence. These sentences are from a specific niche domain -- for example, medical articles associated with pharmaceutical products. You would like to represent each sentence as a sequence of embeddings, where each word token is associated with an embedding. In this case, will you use general purpose word embeddings available for Web corpus, such as pre-trained word2vec vectors or glove vectors? Or will you create corpus specific embeddings? Justify the rationale for your choice.
7. In the previous question, your friend suggests that instead of using word embeddings as representations for the input word tokens, given the computational cost of creating them, you should just use one hot vector encoding of tokens. What are the advantages and disadvantages of this approach?
8. For the question in (7), another friend suggests that instead of using a sequence of pre-trained word embeddings, you will use sentence representation. What is the sentence representation you would use? And why?
9. For the question in (6), you had used a simple neural network consisting of one layer LSTM, followed by a feed forward layer and a softmax layer. Now your friend suggests that instead of using your hand-built neural network, you should use BERT (bidirectional encoder representation from transformers). With BERT, you have two options: You can use it in the fine tuning mode, where you use the domain specific